Python Set

یک **Set**مجموعه‌ای از چند مقدار است که ترتیب مشخصی ندارد ، عضوها Index ندارند. ، مقدار تکراری را قبول نمی‌کند ، می‌توان عضو جدید به آن اضافه کرد ، می‌توان عضوهای آن را حذف کرد ، خود اعضای Set باید از نوعی باشند که قابل hash شدن باشند؛ مثلاً listنمی‌تواند مستقیماً عضو Set باشد

مثال:

```
myset = {"apple", "banana", "cherry"}

print(myset)
```

ممکن است خروجی به شکل زیر باشد:

```
{'banana', 'apple', 'cherry'}
```

ترتیب را نباید ملاک قرار داد

ساختSet

برای ساخت Set از {}استفاده می‌کنیم:

```
fruits = {"apple", "banana", "cherry"}

print(fruits)
```

نکته مهم : این{} = fruits ، یک Setخالی نیست؛ بلکه یک Dictionary خالی است ،برای ساخت Set خالی باید بنویسیم:

```
fruits = set()
```

Setمقدار تکراری ندارد

اگر یک مقدار را چند بار قرار دهیم، فقط یک بار ذخیره می‌شود:

```
fruits = {"apple", "banana", "apple", "cherry", "banana"}

print(fruits)
```

خروجی:

```
{'apple', 'banana', 'cherry'}
```

بنابراین یکی از کاربردهای بسیار مهمSet ، حذف داده‌های تکراری است

مثلاً:

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)
print(unique_numbers)
```

خروجی:

```
{1, 2, 3, 4, 5}
```

Setدارای Index نیست

در List می‌توانیم بنویسیم:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
```

اما در Set نمی‌توانیم:

```
fruits = {"apple", "banana", "cherry"}
print(fruits[0])
```

این کد خطا می‌دهد ،چرا؟ چون Set Unindexed است

Setترتیب مشخصی ندارد

مثلاً:

```
fruits = {"apple", "banana", "cherry"}
print(fruits)
```

ممکن است خروجی:

```
{'apple', 'banana', 'cherry'}
```

یا ترتیب دیگری باشد بنابراین نباید روی ترتیب Set حساب کنیم

تعداد اعضایSet

برای به‌دست آوردن تعداد اعضا از ()lenاستفاده می‌کنیم:

```
fruits = {"apple", "banana", "cherry"}
print(len(fruits))
```

خروجی:

```
3
```

اگر مقدار تکراری داشته باشیم:

```
fruits = {"apple", "banana", "apple", "cherry"}
print(len(fruits))
```

خروجی:

```
3
```

چون appleفقط یک بار در Set نگهداری می‌شود

انواع داده درSet

Setمی‌تواند شاملString ، Integerو Boolean باشد:

```
myset = {"Ali", 20, True}
print(myset)
```

حتی می‌توانیم انواع مختلف را با هم داشته باشیم:

```
myset = {"Ali", 25, 3.14, True}
```

یک نکته جالب درباره True و ۱

درSet ، این دو مقدار برابر در نظر گرفته می‌شوند True و 1

مثلاً:

```
myset = {True, 1, 2}
print(myset)
```

تعداد اعضا 2خواهد بود

به همین شکل:

```
False
```

0

برابر در نظر گرفته می‌شوند:

```
myset = {False, 0, 1}
print(myset)
```

بررسی نوعSet

با type()می‌توانیم نوع داده را بررسی کنیم:

```
myset = {"apple", "banana", "cherry"}
print(type(myset))
```

خروجی:

```
<class 'set'>
```

ساخت Set باset()

می‌توانیم از set()نیز استفاده کنیم:

```
myset = set(("apple", "banana", "cherry"))
print(myset)
```

همچنین یک کاربرد بسیار مهم:

```
numbers = [1, 2, 2, 3, 3, 4]
numbers = set(numbers)
print(numbers)
```

اینجا List را به Set تبدیل کرده‌ایم

ویژگی	List	Tuple	Set	Dictionary
Syntax	[]	()	{}	{key:value}
Ordered	✓	✓	✕	✓
Changeable	✓	✕	✓	✓
Index	✓	✓	✕	باKey
Duplicate	✓	✓	✕	Key ✕
مثال	[1,2,3]	(1,2,3)	{1,2,3}	{"a":1}

یک اصلاح مهم

Setخودش قابل تغییر است، اما نمی‌توانیم مقدار یک عضو موجود را مستقیماً تغییر دهیم.

مثلاً این امکان وجود دارد:

```
fruits = {"apple", "banana", "cherry"}
fruits.add("orange")
```

و:

```
fruits.remove("banana")
```

اما نمی‌توانیم مثل List بنویسیم:

```
fruits[0] = "orange"
```

چون Set دارای index نیست

مثال

```
students = {"Ali", "Sara", "Reza", "Ali", "Sara"}
print(students)
print(len(students))
```

چند دانش‌آموز داریم؟

اگر انتظار داشته باشند جواب ۵ باشد، اینجا می‌توانید توضیح دهید که Set داده‌های تکراری را حذف می‌کند و تعداد اعضای منحصربه‌فرد را نگه می‌دارد

تمرین کلاسی

```
numbers = [10, 20, 20, 30, 40, 40, 50, 50]
numbers_set = set(numbers)
print(numbers_set)
print(len(numbers_set))
```

سؤال:

۱. چند مقدار در List وجود دارد؟

۲. چند مقدار منحصربه‌فرد وجود دارد؟

۳. چرا تعداد اعضای numbers_setکمتر است؟

۴. چرا نمی‌توانیم numbers_set[0]را بنویسیم؟

Access Set Items

در Set برخلاف List نمی‌توانیم با استفاده از Indexبه یک عضو دسترسی پیدا کنیم مثلاً این کد اشتباه است:

```
fruits = {"apple", "banana", "cherry"}

print(fruits[0])
```

خطا می‌دهد، زیرا Set دارای Index نیست اما دو روش مهم برای کار با اعضای Set داریم:

۱. استفاده از for

۲. استفاده از inو not in

دسترسی به اعضای Set باfor

می‌توانیم با حلقه forتمام اعضای Set را یکی‌یکی دریافت کنیم:

```
fruits = {"apple", "banana", "cherry"}

for fruit in fruits:
  print(fruit)
```

خروجی ممکن است به صورت زیر باشد:

```
apple

banana

cherry
```

اما توجه کنید که ترتیب خروجی تضمین‌شده نیست ممکن است Python خروجی را به شکل دیگری نمایش دهد:

```
cherry

apple

banana
```

بررسی وجود یک مقدار باin

با inمی‌توانیم بررسی کنیم که آیا یک مقدار در Set وجود دارد یا خیر

```
fruits = {"apple", "banana", "cherry"}

print("banana" in fruits)
```

خروجی:

```
True
```

چون bananaدر Set وجود دارد

مثال دیگر:

```
print("orange" in fruits)
```

خروجی:

```
False
```

استفاده ازnot in

اگر بخواهیم بررسی کنیم که یک مقدار وجود ندارد، از not inاستفاده می‌کنیم:

```
fruits = {"apple", "banana", "cherry"}

print("orange" not in fruits)
```

خروجی:

```
True
```

چون orangeدر Set وجود ندارد

مثال:

```
print("banana" not in fruits)
```

خروجی:

```
False
```

چون bananaوجود دارد

استفاده از in در if

یکی از کاربردهای مهم inاستفاده از آن در شرط ifاست

```
fruits = {"apple", "banana", "cherry"}
if "banana" in fruits:
   print("Banana is available")
```

خروجی:

```
Banana is available
```

مثال دیگر:

```
fruits = {"apple", "banana", "cherry"}
if "orange" not in fruits:
   print("Orange is not available")
```

خروجی:

```
Orange is not available
```

مثال: بررسی نام دانش‌آموز

فرض کنید لیستی از دانش‌آموزان داریم:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
name = input("Enter your name: ")
if name in students:
   print("You are registered.")
else:
   print("You are not registered.")
```

اگر کاربر وارد کند:

```
Ali
```

خروجی:

You are registered

مثال: بررسی محصول

```
products = {"laptop", "mouse", "keyboard", "monitor"}
product = input("Enter product name: ")
if product in products:
   print("Product is available.")
else:
   print("Product is not available.")
```

این مثال می‌تواند ارتباط خوبی بین Set + input + if ایجاد کند

شمارش اعضای Set با()len

می‌توانیم تعداد اعضای Set را نیز بررسی کنیم:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
print(len(students))
```

خروجی:

```
4
```

مثال:

```
students = {"Ali", "Sara", "Reza", "Ali"}
print(len(students))
```

خروجی:

```
3
```

چون Ali دو بار نوشته شده، ولی Set آن را فقط یک بار نگه می‌دارد

ترکیب for وif

می‌توانیم اعضای Set را بررسی کنیم:

```
numbers = {10, 15, 20, 25, 30}
```

```
for number in numbers:
   if number > 20:
      print(number)
```

خروجی شامل موارد زیر است :

25

30

Change Items

بعد از ساخته شدنSet ، نمی‌توانیم مقدار یک عضو موجود را مستقیماً تغییر دهیم

مثلاً:

```
fruits = {"apple", "banana", "cherry"}
fruits[0] = "orange"
```

این کد اشتباه است

چون:Set

- Indexندارد
- نمی‌توانیم یک عضو را با Index تغییر دهیم

اما می‌توانیم عضو جدید اضافه کنیم یا عضو موجود را حذف کنیم

مثلاً:

```
fruits = {"apple", "banana", "cherry"}
fruits.add("orange")
print(fruits)
```

حالا orangeبه Set اضافه شده است

تمرین ۱ -چاپ اعضا

Setزیر را ایجاد کنید و با forتمام اعضای آن را چاپ کنید:

```
colors = {"red", "green", "blue", "yellow"}
```

تمرین ۲ - بررسی وجود

برنامه‌ای بنویسید که بررسی کند آیا "Python"در Set زیر وجود دارد یا خیر:

```
languages = {"Python", "Java", "C++", "JavaScript"}
```

خروجی مناسب نمایش دهید

تمرین ۳ - استفاده ازin not

برنامه‌ای بنویسید که بررسی کند آیا "PHP"در Set زیر وجود ندارد:

```
languages = {"Python", "Java", "C++", "JavaScript"}
```

اگر وجود نداشت، چاپ کنید:

```
PHP is not available
```

تمرین ۴ - دانش‌آموزان

Setزیر را ایجاد کنید:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
```

نام دانش‌آموز را با ()inputدریافت کنید و بررسی کنید که آیا دانش‌آموز در کلاس وجود دارد یا خیر

تمرین ۵ - محصولات فروشگاه

```
products = {"laptop", "mouse", "keyboard", "monitor"}
```

نام محصول را از کاربر دریافت کنید

اگر محصول وجود داشت:

```
Product is available
```

و اگر وجود نداشت:

Product is not available

تمرین ۶ - اعداد

Setزیر را داریم:

numbers = {10, 20, 30, 40, 50}

با استفاده از forفقط اعدادی را چاپ کنید که بزرگ‌تر از 25هستند

تمرین ۷ - حذف تکراری‌ها

Listزیر را داریم:

numbers = [10, 20, 20, 30, 40, 40, 50, 50]

آن را به Set تبدیل کنید و نتیجه را چاپ کنید

هدف :حذف اعداد تکراری

تمرین ۸ - چالش

برنامه‌ای بنویسید که Set زیر را داشته باشد:

users = {"ali", "sara", "reza", "maryam"}

نام کاربر را با ()inputدریافت کنید اگر کاربر وجود داشت، بنویسید:

Welcome!

در غیر این صورت:

User not found!

تمرین ۹ - چالش

برنامه‌ای بنویسید که:

۱. یک Set از اعداد داشته باشد
۲. با forاعداد را بررسی کند
۳. فقط اعداد زوج را چاپ کند

مثلاً:

numbers = {1, 2, 3, 4, 5, 6, 7, 8}

خروجی:

2

4

6

8

تمرین ۱۰ - پروژه کوچک

یک برنامه ساده برای بررسی موجودی فروشگاه بنویسید

products = {"laptop", "phone", "tablet", "mouse", "keyboard"}

کاربر نام محصول را وارد کند

برنامه باید:

اگر محصول موجود بود → Product is available

اگر موجود نبود → Product is not available

در پایان تعداد کل محصولات موجود را نیز نمایش دهد

مثلاً:

Enter product: phone

Product is available

Total products: 5

Add Set Items

بعد از ایجاد یکSet ، نمی‌توانیم مقدار یک عضو موجود را مستقیماً تغییر دهیم؛ اما می‌توانیم عضو جدید اضافه کنیم

برای اضافه کردن عضو به Set دو روش مهم داریم:

Method	کاربرد
add()	اضافه کردن **یک عضو**
update()	اضافه کردن **چند عضو**

استفاده از ()add

اگر بخواهیم **یک مقدار** به Set اضافه کنیم، از ()addاستفاده می‌کنیم

```
fruits = {"apple", "banana", "cherry"}

fruits.add("orange")

print(fruits)
```

بعد از اجرای برنامه، orangeبه Set اضافه می‌شود

مثلاً:

```
{'apple', 'banana', 'cherry', 'orange'}
```

ترتیب نمایش اعضای Set تضمین‌شده نیست

اضافه کردن یک مقدار تکراری

اگر مقداری که می‌خواهیم اضافه کنیم از قبل در Set وجود داشته باشد، اتفاق خاصی نمی‌افتد

```
fruits = {"apple", "banana", "cherry"}

fruits.add("apple")

print(fruits)
```

خروجی همچنان شامل یک appleخواهد بود ، چرا؟

چون Setاجازه‌ی مقدار تکراری نمی‌دهد

مثال با اعداد

```
numbers = {10, 20, 30}

numbers.add(40)

print(numbers)
```

خروجی:

{10, 20, 30, 40}

مثال دیگر:

```
numbers.add(50)
numbers.add(60)
print(numbers)
```

حالا Set شامل اعداد بیشتری است

اضافه کردن مقدار از کاربر

می‌توانیم مقدار جدید را با input()دریافت کنیم:

```
fruits = {"apple", "banana", "cherry"}
fruit = input("Enter a fruit: ")
fruits.add(fruit)
print(fruits)
```

اگر کاربر وارد کند:

```
orange
```

orangeبه Set اضافه می‌شود

استفاده از update()

اگر بخواهیم چند مقدار را به Set اضافه کنیم، از update()استفاده می‌کنیم

مثلاً:

```
fruits = {"apple", "banana", "cherry"}
new_fruits = {"orange", "mango", "kiwi"}
fruits.update(new_fruits)
print(fruits)
```

در اینجا تمام اعضای new_fruitsبه fruitsاضافه می‌شوند

تفاوت add() و update()

add()

یک عضو اضافه می‌کند:

```
fruits.add("orange")
```

update()

چند عضو را اضافه می‌کند:

```
fruits.update(["orange", "mango", "kiwi"])
```

بنابراین:

add() → یک item

update() → چند item

استفاده از update() باList

update()فقط برای Set نیست می‌توانیم یک List را نیز به Set اضافه کنیم:

```
fruits = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]
fruits.update(mylist)
print(fruits)
```

در نتیجه kiwiو orangeبه Set اضافه می‌شوند

استفاده از update() باTuple

می‌توانیم Tuple را نیز استفاده کنیم:

```
fruits = {"apple", "banana"}
new_fruits = ("orange", "mango")
fruits.update(new_fruits)
print(fruits)
```

استفاده از update() با List وSet

مثلاً:

```
fruits = {"apple", "banana"}
mylist = ["orange", "mango"]
myset = {"kiwi", "pineapple"}
fruits.update(mylist)
fruits.update(myset)
print(fruits)
```

هر دو مجموعه به fruitsاضافه می‌شوند

یک نکته بسیار مهم

در update()، Pythonاعضای Iterable را یکی‌یکی به Set اضافه می‌کند

مثلاً:

```
fruits = {"apple"}
fruits.update(["banana", "orange"])
print(fruits)
```

نتیجه:

```
{'apple', 'banana', 'orange'}
```

اما اگر از add()استفاده کنیم:

```
fruits = {"apple"}
fruits.add(("banana", "orange"))
print(fruits)
```

اینجا خود Tuple به عنوان یک عضو اضافه می‌شود:

```
{'apple', ('banana', 'orange')}
```

9. update() با Dictionary

می‌توانیم Dictionary را نیز به update() بدهیم:

```
fruits = {"apple", "banana"}
mydict = {
   "a": "apple",
   "b": "banana",
   "c": "cherry"
}
fruits.update(mydict)
print(fruits)
```

در این حالت Keyهای Dictionary به Set اضافه می‌شوند:

```
a
b
c
```

یعنی:

```
fruits.update(mydict)
```

عملاً Key های mydict را وارد Set می‌کند

مثال نهایی:

```
fruits = {"apple", "banana"}
# Add one item
fruits.add("orange")
# Add multiple items
fruits.update(["mango", "kiwi"])
print(fruits)
```

تمرین ۱

Setزیر را ایجاد کنید:

```
colors = {"red", "green", "blue"}
```

رنگ yellowرا با استفاده از ()addاضافه کنید

تمرین ۲

```
numbers = {10, 20, 30}
```

اعداد 40و 50را به Set اضافه کنید **نکته،** از ()addاستفاده کنید

تمرین ۳

Setزیر را دارید:

```
fruits = {"apple", "banana"}
```

با استفاده از ()updateموارد زیر را اضافه کنید:

```
["orange", "mango", "kiwi"]
```

تمرین ۴

کد زیر را کامل کنید:

```
students = {"Ali", "Sara", "Reza"}
name = input("Enter student name: ")
# Add name to students
```

نام واردشده توسط کاربر را به Set اضافه کنید

تمرین ۵

دو Set داریم:

```
set1 = {"Python", "Java"}
set2 = {"C++", "JavaScript", "PHP"}
```

تمام اعضای set2را به set1اضافه کنید خروجی باید شامل تمام زبان‌ها باشد

تمرین ۶

Listزیر را داریم:

```
numbers = {10, 20, 30}
new_numbers = [40, 50, 60, 70]
```

با استفاده از ()updateتمام اعداد new_numbersرا به numbersاضافه کنید

تمرین ۷ - تفاوت add وupdate

پیش‌بینی کنید خروجی هر کدام چیست:

کد اول:

```
myset = {"apple"}
myset.add(("banana", "orange"))
print(myset)
```

کد دوم:

```
myset = {"apple"}
myset.update(("banana", "orange"))
print(myset)
```

چرا خروجی این دو برنامه متفاوت است؟

تمرین ۸ - حذف داده‌های تکراری

Listزیر را داریم:

```
numbers = [10, 20, 20, 30, 40, 40, 50]
```

آن را به Set تبدیل کنید و سپس عدد 60را با ()addاضافه کنید

تمرین ۹ - سیستم ثبت دانش‌آموز

برنامه‌ای بنویسید که:

```
students = {"Ali", "Sara", "Reza"}
```

نام یک دانش‌آموز جدید را از کاربر دریافت کند و به Set اضافه کند

در پایان:

۱. تمام دانش‌آموزان را چاپ کند

۲. تعداد دانش‌آموزان را با len()نمایش دهد

تمرین ۱۰ - سیستم موجودی فروشگاه

یک Set برای محصولات فروشگاه ایجاد کنید:

```
products = {"laptop", "mouse", "keyboard"}
```

سپس:

۱. یک محصول جدید با input()از کاربر بگیرید

۲. آن را با add()اضافه کنید

۳. سه محصول جدید را با update()اضافه کنید

۴. در پایان محصولات را چاپ کنید

۵. تعداد محصولات را با len()نمایش دهید

add()برای اضافه کردن یک Item است، اما update()برای اضافه کردن چند Item است

مثال:

```
myset.add("apple")
```

در مقابل:

```
myset.update(["apple", "banana", "orange"])
```

Remove Set Items

در Python چند روش برای حذف اطلاعات از یک Set وجود دارد:

Method / Keyword	کاربرد
remove()	حذف یک عضو مشخص؛ اگر وجود نداشته باشد → Error
discard()	حذف یک عضو مشخص؛ اگر وجود نداشته باشد ⟶ بدون Error
pop()	حذف یک عضو نامشخص و برگرداندن آن
clear()	حذف تمام اعضای Set
del	حذف خود Set

استفاده از remove()

برای حذف یک عضو مشخص از Set می‌توانیم از remove()استفاده کنیم

```
fruits = {"apple", "banana", "cherry"}

fruits.remove("banana")

print(fruits)
```

بعد از اجرای برنامه، banana حذف می‌شود:

```
{'apple', 'cherry'}
```

البته ترتیب نمایش Set تضمین‌شده نیست

اگر عضو وجود نداشته باشد

این نکته بسیار مهم است:

```
fruits = {"apple", "banana", "cherry"}

fruits.remove("orange")
```

چون orangeدر Set وجود ندارد، Pythonخطا می‌دهد:

```
KeyError
```

استفاده از discard()

روش دیگری برای حذف عضو، discard()است:

```
fruits = {"apple", "banana", "cherry"}

fruits.discard("banana")

print(fruits)
```

bananaحذف می‌شود

تفاوت مهم remove وdiscard

اگر عضو وجود داشته باشد، هر دو تقریباً یک کار انجام می‌دهند:

```
fruits.remove("banana")
```

یا:

```
fruits.discard("banana")
```

اما اگر عضو وجود نداشته باشد:

```
fruits.remove("orange")
```

Error

ولی:

```
fruits.discard("orange")
```

هیچ خطایی ایجاد نمی‌کند

خلاصه:

discard()	remove()	وضعیت
حذف می‌کند	حذف می‌کند	عضو وجود دارد
کاری نمی‌کند	Error	عضو وجود ندارد

چه زمانی از remove و discard استفاده کنیم؟

اگر مطمئن هستیم عضو وجود دارد (fruits.remove("banana") مناسب است

اما اگر نمی‌دانیم عضو وجود دارد یا نه، discard()انتخاب امن‌تری است:

```
fruits.discard("orange")
```

استفاده از pop()

روش دیگری برای حذف عضو ازSet ، pop()است:

```
fruits = {"apple", "banana", "cherry"}
fruits.pop()
print(fruits)
```

اما یک نکته بسیار مهم وجود دارد: ()popعضو مشخصی را حذف نمی‌کند چون Set unordered است، نمی‌توانیم مشخص کنیم کدام عضو حذف شود

دریافت عضو حذف‌شده با()pop

یکی از ویژگی‌های مهم ()popاین است که مقدار حذف‌شده را برمی‌گرداند

مثلاً:

```
fruits = {"apple", "banana", "cherry"}

x = fruits.pop()

print("Removed:", x)

print("Remaining:", fruits)
```

مثلاً ممکن است خروجی باشد:

```
Removed: banana

Remaining: {'apple', 'cherry'}
```

اما ممکن است عضو دیگری حذف شود بنابراین نباید برنامه را طوری طراحی کنیم که انتظار داشته باشد همیشه مثلاً banana حذف شود

تفاوت remove وpop

این دو را با هم مقایسه کنید:

()remove

```
fruits.remove("banana")
```

یعنی bananaرا حذف کن

()pop

```
x = fruits.pop()
```

یعنی یک عضو نامشخص را حذف کن و آن را به من برگردان

استفاده از ()clear

اگر بخواهیم تمام اعضای Set را حذف کنیم، از ()clearاستفاده می‌کنیم:

```
fruits = {"apple", "banana", "cherry"}
fruits.clear()
print(fruits)
```

خروجی:

```
set()
```

توجه کنید که Set هنوز وجود دارد، اما خالی شده است

یعنی قبل:

```
{"apple", "banana", "cherry"}
clear()
```

بعد:

```
set()
```

تفاوت clear() وdel

این تفاوت بسیار مهم است

clear()

محتوای Set را پاک می‌کند:

```
fruits = {"apple", "banana", "cherry"}
fruits.clear()
print(fruits)
```

خروجی:

```
set()
```

Setهنوز وجود دارد

del

خود متغیر Set را حذف می‌کند:

```
fruits = {"apple", "banana", "cherry"}

del fruits
```

حالا اگر بنویسیم print(fruits) خطا می‌گیریم، چون متغیر fruitsدیگر وجود ندارد

clear() vs del

دستور	چه چیزی حذف می‌شود؟
remove()	یک عضو مشخص
discard()	یک عضو مشخص
pop()	یک عضو نامشخص
clear()	تمام اعضا
del	خود Set / متغیر

مثال ۱ - حذف یک دانش‌آموز

```
students = {"Ali", "Sara", "Reza", "Maryam"}

students.remove("Reza")

print(students)
```

دانش‌آموز Rezaاز Set حذف می‌شود

مثال کاربردی ۲ - حذف امن

فرض کنید نمی‌دانیم دانش‌آموز وجود دارد یا نه:

```
students = {"Ali", "Sara", "Reza", "Maryam"}

name = input("Enter student name: ")

students.discard(name)

print(students)
```

اگر کاربر وارد کند:

Reza

Rezaحذف می‌شود و اگر وارد کند Hassan هیچ خطایی ایجاد نمی‌شود

مثال ۳ - حذف چند عضو

می‌توانیم چند بار ()removeیا ()discardرا اجرا کنیم:

```
fruits = {"apple", "banana", "cherry", "orange"}
fruits.remove("banana")
fruits.remove("orange")
print(fruits)
```

مثال ۴ - حذف تمام اطلاعات

```
students = {"Ali", "Sara", "Reza"}
students.clear()
print(students)
```

خروجی:

```
set()
```

مثال ۵ - بررسی قبل از remove

اگر بخواهیم از خطای ()removeجلوگیری کنیم، می‌توانیم ابتدا با inبررسی کنیم:

```
fruits = {"apple", "banana", "cherry"}
if "orange" in fruits:
   fruits.remove("orange")
else:
   print("Orange does not exist.")
```

تمرین ۱

Setزیر را ایجاد کنید:

colors = {"red", "green", "blue", "yellow"}

رنگ greenرا با استفاده از ()removeحذف کنید

تمرین ۲

fruits = {"apple", "banana", "cherry"}

با استفاده از ()discardمقدار bananaرا حذف کنید

تمرین ۳

کد زیر را اجرا کنید:

```
fruits = {"apple", "banana", "cherry"}
fruits.remove("orange")
```

چه اتفاقی می‌افتد؟

سپس برنامه را با ()discardاصلاح کنید

تمرین ۴

Setزیر را داریم:

numbers = {10, 20, 30, 40, 50}

عدد 30را حذف کنید

تمرین ۵

Setزیر را داریم:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
```

نام یک دانش‌آموز را با ()inputدریافت کنید و با استفاده از ()discardآن را حذف کنید

در پایان Set را چاپ کنید

تمرین ۶

برنامه‌ای بنویسید که قبل از حذف بررسی کند آیا bananaوجود دارد یا خیر

fruits = {"apple", "banana", "cherry"}

اگر وجود داشت:

Banana removed

در غیر این صورت:

Banana not found

تمرین ۷–pop()

کد زیر را اجرا کنید:

```
fruits = {"apple", "banana", "cherry"}
x = fruits.pop()
print("Removed:", x)
print(fruits)
```

مقدار xچیست؟

آیا می‌توانید مطمئن باشید همیشه appleحذف می‌شود؟

تمرین ۸–clear()

Setزیر را کاملاً خالی کنید:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
```

در پایان باید داشته باشید:

set()

تمرین ۹ - مدیریت محصولات

یک Set از محصولات فروشگاه ایجاد کنید:

```
products = {"laptop", "mouse", "keyboard", "monitor"}
```

نام محصول را از کاربر دریافت کنید

اگر محصول وجود داشت، آن را حذف کنید و پیام زیر را نمایش دهید:

Product removed successfully

اگر وجود نداشت:

Product not found

تمرین ۱۰ - پروژه کوچک

برنامه‌ای برای مدیریت لیست دانش‌آموزان بنویسید:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
```

برنامه از کاربر نام دانش‌آموز را بگیرد و بپرسد:

```
Enter student name:
```

اگر دانش‌آموز وجود داشت، او را حذف کند

در پایان:

۱. لیست دانش‌آموزان باقی‌مانده را نمایش دهد

۲. تعداد دانش‌آموزان باقی‌مانده را با len()نمایش دهد

تمرین

به نظر شما خروجی این برنامه چیست؟

```
myset = {"apple", "banana", "cherry"}
myset.remove("banana")
myset.discard("orange")
x = myset.pop()
print(x)
print(myset)
```

سؤال :چرا نمی‌توانیم با اطمینان بگوییم مقدار xچیست؟

Loop Sets

از آنجا که Setدارای Index نیست، نمی‌توانیم با روش زیر به اعضای آن دسترسی پیدا کنیم:

```
fruits = {"apple", "banana", "cherry"}
print(fruits[0])
```

این کد خطا می‌دهد

برای دسترسی به تمام اعضای Set معمولاً از حلقه forاستفاده می‌کنیم

استفاده از for برای پیمایشSet

ساختار کلی:

```
for variable in set:
   # code
```

مثال:

```
fruits = {"apple", "banana", "cherry"}
for fruit in fruits:
   print(fruit)
```

خروجی ممکن است مثلاً این باشد:

```
apple
banana
cherry
```

اما ممکن است ترتیب متفاوت باشد:

```
banana
cherry
apple
```

چرا؟ چون Set Unordered است بنابراین نباید انتظار داشته باشیم اعضا همیشه با یک ترتیب مشخص چاپ شوند

نام متغیر در حلقه

در این مثال:

```
for fruit in fruits:
   print(fruit)
```

fruitیک متغیر موقت است که در هر مرحله یکی از اعضای Set را دریافت می‌کند

مثلاً:

مرحله ۱ → fruit = "apple"

مرحله ۲ → fruit = "banana"

مرحله ۳ → fruit = "cherry"

سپس print()مقدار آن را نمایش می‌دهد

استفاده از نام دیگر برای متغیر

اسم متغیر مهم نیست

مثلاً:

```
fruits = {"apple", "banana", "cherry"}
for x in fruits:
  print(x)
```

یا:

```
for item in fruits:
  print(item)
```

هر دو درست هستند

اما بهتر است اسم متغیر معنادار باشد:

```
for fruit in fruits:
```

بهتر از:

```
for x in fruits:
```

استفاده از if داخل حلقه

می‌توانیم همراه با forاز ifنیز استفاده کنیم

مثلاً:

```
numbers = {10, 15, 20, 25, 30}
for number in numbers:
    if number > 20:
        print(number)
```

خروجی:

```
25
30
```

در اینجا Python تمام اعضای Set را بررسی می‌کند و فقط اعدادی را چاپ می‌کند که بزرگ‌تر از 20هستند

چاپ اعداد زوج

```
numbers = {1, 2, 3, 4, 5, 6, 7, 8}
for number in numbers:
    if number % 2 == 0:
        print(number)
```

خروجی:

```
2
4
6
8
```

بررسی وجود یک عضو داخل حلقه

مثلاً:

```
fruits = {"apple", "banana", "cherry"}
for fruit in fruits:
    if fruit == "banana":
        print("Banana found!")
```

خروجی:

```
Banana found!
```

البته برای صرفاً بررسی وجود یک عضو، استفاده از in ساده‌تر است:

```
if "banana" in fruits:
   print("Banana found!")
```

مثال- دانش‌آموزان

فرض کنید یک Set از نام دانش‌آموزان داریم:

```
students = {"Ali", "Sara", "Reza", "Maryam"}
for student in students:
   print(student)
```

تمام دانش‌آموزان چاپ می‌شوند

مثال– نمرات

```
scores = {12, 15, 18, 20, 10}
for score in scores:
   if score >= 15:
      print(score)
```

این برنامه فقط نمرات 15 یا بالاتر را نمایش می‌دهد

مثال- محصولات فروشگاه

```
products = {"laptop", "mouse", "keyboard", "monitor"}
for product in products:
   print("Product:", product)
```

خروجی مثلاً:

```
Product: laptop
```

```
Product: mouse
Product: keyboard
Product: monitor
```

ترکیب for وinput

می‌توانیم از کاربر یک محصول دریافت کنیم و با Set بررسی کنیم:

```
products = {"laptop", "mouse", "keyboard", "monitor"}
product = input("Enter product: ")
for item in products:
    if item == product:
        print("Product found!")
```

اما این روش کمی طولانی است

روش بهتر:

```
if product in products:
    print("Product found!")
```

نکته مهم Set :و List در حلقه

هر دو را می‌توان با forپیمایش کرد:

List:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Set:

```
fruits = {"apple", "banana", "cherry"}
for fruit in fruits:
    print(fruit)
```

تفاوت مهم این است که Listترتیب دارد ولی Set ترتیب مشخصی ندارد

مقایسه

ویژگی	List	Set
for	✓	✓
Index	✓	✗
ترتیب	✓	✗
Duplicate	✓	✗
in	✓	✓

تمرین ۱

Setزیر را ایجاد کنید:

```
colors = {"red", "green", "blue", "yellow"}
```

با استفاده از forتمام رنگ‌ها را چاپ کنید

تمرین ۲

```
numbers = {10, 20, 30, 40, 50}
```

با استفاده از forتمام اعداد را چاپ کنید

تمرین ۳

برنامه‌ای بنویسید که فقط اعداد بزرگ‌تر از 25را چاپ کند:

```
numbers = {10, 20, 30, 40, 50}
```

خروجی:

30

40

50

تمرین ۴

Setزیر را دارید:

numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

با استفاده از forفقط اعداد زوج را چاپ کنید

تمرین ۵

Setزیر را دارید:

students = {"Ali", "Sara", "Reza", "Maryam"}

با استفاده از forنام تمام دانش‌آموزان را به صورت زیر چاپ کنید:

Student: Ali

Student: Sara

...

تمرین ۶

Setزیر را دارید:

scores = {8, 12, 15, 17, 19, 20}

با استفاده از forفقط نمراتی را چاپ کنید که بزرگ‌تر یا مساوی ۱۵ هستند

تمرین ۷

Setزیر را دارید:

numbers = {5, 10, 15, 20, 25, 30}

با استفاده از forاعدادی را چاپ کنید که بر 5بخش‌پذیر هستند

تمرین ۸

برنامه‌ای بنویسید که Set زیر را پیمایش کند:

fruits = {"apple", "banana", "orange", "mango"}

اگر میوه bananaبود، پیام زیر را چاپ کند:

Banana found!

تمرین ۹ - جمع اعضای Set

Setزیر را داریم:

numbers = {10, 20, 30, 40, 50}

با استفاده از forمجموع تمام اعداد را محاسبه کنید

خروجی:

Total: 150

راهنمایی:

یک متغیر برای نگهداری مجموع ایجاد کنید:

total = 0

تمرین ۱۰ - شمارش اعداد زوج

Setزیر را داریم:

numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

با استفاده از forتعداد اعداد زوج را محاسبه کنید

خروجی:

Even numbers: 5

تمرین ۱۱- سیستم دانش‌آموزان

Setزیر را داریم:

students = {"Ali", "Sara", "Reza", "Maryam"}

برنامه‌ای بنویسید که:

۱. تمام دانش‌آموزان را با forچاپ کند
۲. از کاربر نام یک دانش‌آموز را دریافت کند
۳. با استفاده از inبررسی کند که آیا دانش‌آموز در Set وجود دارد یا خیر

۴. پیام مناسب نمایش دهد

تمرین ۱۲ - پروژه فروشگاه

یک Set برای محصولات فروشگاه ایجاد کنید:

```
products = {"laptop", "mouse", "keyboard", "monitor", "phone"}
```

برنامه باید:

۱. تمام محصولات را با forنمایش دهد

۲. نام یک محصول را از کاربر دریافت کند

۳. با inبررسی کند محصول وجود دارد یا خیر

۴. اگر وجود داشت، پیام Product is availableرا نمایش دهد

۵. اگر وجود نداشت، پیام Product is not availableرا نمایش دهد

چالش

برنامه زیر را کامل کنید:

```
numbers = {10, 15, 20, 25, 30, 35, 40}
even_count = 0
odd_count = 0
for number in numbers:
    # Complete the code
```

برنامه باید در پایان تعداد اعداد زوج و فرد را نمایش دهد

خروجی مورد انتظار:

```
Even numbers: 4
Odd numbers: 3
```

Python – Join Sets

در Python می‌توانیم دو یا چند Set را با روش‌های مختلف با هم ترکیب یا مقایسه کنیم

چهار عملیات اصلی:

Operation	Method	Operator	مفهوم
Union	union()	|	همه اعضای دوSet
Intersection	intersection()	&	اعضای مشترک
Difference	difference()	-	اعضای Set اول که در دوم نیستند
Symmetric Difference	symmetric_difference()	^	اعضایی که فقط در یکی از دو Set هستند

Unionاجتماع دوSet

()unionتمام اعضای دو Set را در یک Set جدید قرار می‌دهد

مثلاً:

```
set1 = {"a", "b", "c"}

set2 = {1, 2, 3}

set3 = set1.union(set2)

print(set3)
```

نتیجه شامل تمام اعضای هر دو Set است:

```
{'a', 'b', 'c', 1, 2, 3}
```

البته ترتیب نمایش ممکن است متفاوت باشد

استفاده از |

به جای ()unionمی‌توانیم از عملگر |استفاده کنیم:

```
set1 = {"a", "b", "c"}

set2 = {1, 2, 3}

set3 = set1 | set2
```

```
print(set3)
```

نتیجه همان است

بنابراین:

```
set1.union(set2)
```

و:

```
set1 | set2
```

معادل هستند

UnionچندSet

می‌توانیم چند Set را با هم ترکیب کنیم:

```
set1 = {"a", "b"}
set2 = {"c", "d"}
set3 = {"e", "f"}
result = set1.union(set2, set3)
print(result)
```

یا:

```
result = set1 | set2 | set3
```

Unionبا List وTuple

یک نکته مهم:

()unionمی‌تواند باIterable هایی مثل List و Tuple نیز کار کند

مثلاً:

```
myset = {"apple", "banana"}
mylist = ["orange", "mango"]
result = myset.union(mylist)
```

```
print(result)
```

اما عملگر |فقط باSet ها کار می‌کند:

```
myset | mylist
```

خطا می‌دهد

Updateاضافه کردن به Set اصلی

()updateتمام اعضای Set دوم را به Setاول اضافه می‌کند

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
set1.update(set2)
print(set1)
```

بعد از ()update، خود set1تغییر کرده است

تفاوت مهم

union():

```
result = set1.union(set2)
```

یک Setجدید ایجاد می‌کند

اما :()update

```
set1.update(set2)
```

خود set1را تغییر می‌دهد

Union vs Update

update()	union()
Setاصلی را تغییر می‌دهد	Setجدید ایجاد می‌کند
set1.update(set2)	set3 = set1.union(set2)
set1تغییر می‌کند	set1تغییر نمی‌کند

مثال:

```
set1 = {"apple", "banana"}
set2 = {"orange"}
result = set1.union(set2)
print(set1)
print(result)
```

خروجی:

```
{'apple', 'banana'}
{'apple', 'banana', 'orange'}
```

ولی:

```
set1 = {"apple", "banana"}
set2 = {"orange"}
set1.update(set2)
print(set1)
```

خروجی:

```
{'apple', 'banana', 'orange'}
```

Intersection اشتراک

()intersectionفقط اعضایی را نگه می‌دارد که در هر دو Set وجود دارند

مثلاً:

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}
```

```
result = set1.intersection(set2)
print(result)
```

خروجی:

```
{'apple'}
```

چون فقط appleدر هر دو Set وجود دارد

مثال :

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
result = set1.intersection(set2)
print(result)
```

خروجی:

```
{3, 4}
```

استفاده از &

به جای:

```
set1.intersection(set2)
```

می‌توانیم بنویسیم:

```
set1 & set2
```

مثلاً:

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
result = set1 & set2
print(result)
```

خروجی:

{3, 4}

intersection_update()

اگر بخواهیم نتیجه Intersection مستقیماً روی Set اول اعمال شود، از:

intersection_update()

استفاده می‌کنیم.

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
set1.intersection_update(set2)
print(set1)
```

خروجی:

{3, 4}

یعنی خود set1تغییر کرده است

Difference تفاوت

()differenceاعضایی را نگه می‌دارد که در Setاول هستند ولی در Set دوم نیستند

مثلاً:

```
set1 = {"apple", "banana", "cherry"}
set2 = {"apple", "google", "microsoft"}
result = set1.difference(set2)
print(result)
```

خروجی:

{'banana', 'cherry'}

چون:

set1 = apple, banana, cherry

set2 = apple, google, microsoft

apple = مشترک

set1 = banana, cherry فقط در

Differenceبه کمک

به جای:

```
set1.difference(set2)
```

می‌توانیم بنویسیم:

```
set1 - set2
```

مثلاً:

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
print(set1 - set2)
```

خروجی:

```
{1, 2}
```

ترتیب در Difference مهم است

این دو الزاماً یک نتیجه ندارند:

```
set1 - set2
```

و:

```
set2 - set1
```

مثلاً:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 - set2)
print(set2 - set1)
```

خروجی:

{1, 2}

{4, 5}

در Difference همیشه بپرسید از کدام Set شروع کرده‌ایم؟

difference_update()

این روش مانند difference()است، اما خود Set اول را تغییر می‌دهد.

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
set1.difference_update(set2)
print(set1)
```

خروجی:

{1, 2}

Symmetric Difference

این قسمت کمی سخت‌تر است Symmetric Difference اعضایی را نگه می‌دارد که در یکی ازSet ها وجود دارند، اما در هر دو Set همزمان وجود ندارند

مثال:

```
set1 = {"apple", "banana", "cherry"}
set2 = {"apple", "google", "microsoft"}
result = set1.symmetric_difference(set2)
print(result)
```

خروجی:

{'banana', 'cherry', 'google', 'microsoft'}

چرا؟ apple در هر دو وجود دارد، بنابراین حذف می‌شود بقیه اعضا فقط در یکی از دو Set هستند، پس باقی می‌مانند

استفاده از ^

به جای set1.symmetric_difference(set2) می‌توانیم بنویسیم:

```
set1 ^ set2
```

مثلاً:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 ^ set2)
```

خروجی:

```
{1, 2, 4, 5}
```

symmetric_difference_update()

این روش نتیجه را مستقیماً روی Set اول اعمال می‌کند:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set1.symmetric_difference_update(set2)
print(set1)
```

خروجی:

```
{1, 2, 4, 5}
```

چهار مفهوم اصلی را با یک مثال یاد بگیریم

فرض کنید:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Union

```
A | B
```

نتیجه:

```
{1, 2, 3, 4, 5, 6}
```

Intersection

A & B

نتیجه:

{3, 4}

مشترک‌ها

Difference

A - B

نتیجه:

{1, 2}

فقط A

Symmetric Difference

A ^ B

نتیجه:

{1, 2, 5, 6}

فقط یکی از آنها، نه هر دو

Set Operations

Operation	Operator	نتیجه
Union	|	همه اعضا
Intersection	&	مشترک‌ها
Difference	-	فقط اعضای Set اول
Symmetric Difference	^	غیرمشترک‌ها

مثال ۱ - دانشجویان دو کلاس

کلاس Python:

python_students = {"Ali", "Sara", "Reza", "Maryam"}

کلاس Java:

```
java_students = {"Ali", "Reza", "Hassan", "Nima"}
```

دانشجویان هر دو کلاس:

```
both = python_students & java_students
print(both)
```

نتیجه:

```
{'Ali', 'Reza'}
```

تمام دانشجویان:

```
all_students = python_students | java_students
print(all_students)
```

فقط دانشجویان:Python

```
only_python = python_students - java_students
print(only_python)
```

دانشجویانی که فقط در یکی از کلاس‌ها هستند:

```
one_class = python_students ^ java_students
print(one_class)
```

این مثال برای آموزش مفهوم Set بسیار عالی است

تمرین ۱Union

```
A = {1, 2, 3}
B = {4, 5, 6}
```

با ()unionدو Set را ترکیب کنید

تمرین ۲Intersection

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

اعضای مشترک را پیدا کنید

تمرین ۳Difference

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

فقط اعضایی را پیدا کنید که در Aهستند و در Bنیستند

تمرین ۴Symmetric Difference

A = {1, 2, 3}

B = {3, 4, 5}

اعضایی را پیدا کنید که فقط در یکی از دو Set وجود دارند

تمرین ۵Operator

نتیجه هر کدام را پیش‌بینی کنید:

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

print(A | B)

print(A & B)

print(A - B)

print(A ^ B)

تمرین ۶ - دانشجویان

python = {"Ali", "Sara", "Reza", "Maryam"}

java = {"Ali", "Reza", "Hassan", "Nima"}

پیدا کنید:

۱. دانشجویان هر دو کلاس

۲. تمام دانشجویان

۳. فقط دانشجویان Python

۴. فقط دانشجویان Java

تمرین ۷ Union- یاUpdate ؟

کد زیر را بررسی کنید:

```
A = {1, 2, 3}
B = {3, 4, 5}
C = A.union(B)
print(A)
print(C)
```

آیا Aتغییر کرده است؟

سپس برنامه را با ()updateبنویسید و تفاوت را مشاهده کنید

تمرین ۸ - سهSet

سه کلاس داریم:

```
python = {"Ali", "Sara", "Reza", "Maryam"}
java = {"Ali", "Reza", "Hassan"}
database = {"Ali", "Maryam", "Nima"}
```

سؤال ۱

چه کسانی در Python و Java هستند؟

سؤال ۲

چه کسانی در Python و Database هستند؟

سؤال ۳

تمام دانشجویانی که حداقل در یکی از کلاس‌ها هستند چه کسانی‌اند؟

راهنمایی:

python | java | database

تمرین

دو فروشگاه داریم:

```
shop1 = {"laptop", "mouse", "keyboard", "monitor"}
shop2 = {"laptop", "keyboard", "printer", "scanner"}
```

برنامه‌ای بنویسید که مشخص کند:

۱. چه محصولاتی در هر دو فروشگاه هستند؟
۲. چه محصولاتی در فروشگاه اول هستند ولی در دوم نیستند؟
۳. چه محصولاتی در فروشگاه دوم هستند ولی در اول نیستند؟
۴. چه محصولاتی فقط در یکی از دو فروشگاه وجود دارند؟
۵. تمام محصولات موجود در هر دو فروشگاه چه هستند؟

Python – Frozenset

Frozensetچیست؟

frozensetیک نسخه‌ی **Immutable**از setاست.

یعنی مانند:Set

- اعضای تکراری ندارد .
- ترتیب مشخصی ندارد .
- Indexندارد .
- می‌توان روی آن عملیات مجموعه‌ای انجام داد .

اما یک تفاوت بسیار مهم دارد بعد از ایجاد Frozenset نمی‌توان اعضای آن را اضافه، حذف یا تغییر داد

ساختFrozenset

برای ساخت Frozenset از frozenset()استفاده می‌کنیم:

```
fruits = frozenset({"apple", "banana", "cherry"})
print(fruits)
print(type(fruits))
```

خروجی:

```
frozenset({'apple', 'banana', 'cherry'})
<class 'frozenset'>
```

تفاوت Set وFrozenset

یک Set معمولی:

```
fruits = {"apple", "banana", "cherry"}
```

قابل تغییر است:

```
fruits.add("orange")
```

اما:Frozenset

```
fruits = frozenset({"apple", "banana", "cherry"})
```

قابل تغییر نیست

مثلاً fruits.add("orange") خطا می‌دهد همچنین fruits.remove("apple") خطا می‌دهد

Setدر مقابلFrozenset

ویژگی	set	frozenset
Unique	✓	✓
Ordered	×	×
Index	×	×
قابل تغییر	✓	×
add()	✓	×
remove()	✓	×
discard()	✓	×
clear()	✓	×
union()	✓	✓
intersection()	✓	✓
difference()	✓	✓
symmetric_difference()	✓	✓
قابل استفاده به عنوانDictionary Key	×	✓

چرا Frozenset داریم؟

اگر Set داریم، چرا Frozenset ساخته شده است؟

چون گاهی می‌خواهیم یک مجموعه داشته باشیم که هیچ‌کس نتواند بعداً اعضای آن را تغییر دهد

مثلاً:

```
permissions = frozenset({"read", "write", "execute"})
```

حالا این مجموعه ثابت است و نمی‌توان به آن عضو جدید اضافه کرد

Frozensetوfor

مانند Set می‌توانیم Frozenset را پیمایش کنیم:

```
fruits = frozenset({"apple", "banana", "cherry"})
for fruit in fruits:
    print(fruit)
```

استفاده از len()

مانند Set:

```
fruits = frozenset({"apple", "banana", "cherry"})
print(len(fruits))
```

خروجی:

```
3
```

بررسی وجود با in

می‌توانیم از in استفاده کنیم:

```
fruits = frozenset({"apple", "banana", "cherry"})
print("apple" in fruits)
```

خروجی:

```
True
```

و:

```
print("orange" in fruits)
```

خروجی:

```
False
```

Union در Frozenset

Frozenset از عملیات union() پشتیبانی می‌کند:

```
A = frozenset({1, 2, 3})
B = frozenset({3, 4, 5})
C = A.union(B)
print(C)
```

نتیجه:

```
frozenset({1, 2, 3, 4, 5})
```

نکته مهم:

A و B تغییر نکرده‌اند

Intersection

می‌توانیم اعضای مشترک را پیدا کنیم:

```
A = frozenset({1, 2, 3, 4})
B = frozenset({3, 4, 5, 6})
C = A.intersection(B)
print(C)
```

نتیجه:

```
frozenset({3, 4})
```

یا با :&

```
C = A & B
```

Difference

```
A = frozenset({1, 2, 3, 4})
B = frozenset({3, 4, 5, 6})
C = A.difference(B)
print(C)
```

نتیجه:

```
frozenset({1, 2})
```

یا:

```
C = A – B
```

Symmetric Difference

```
A = frozenset({1, 2, 3})
B = frozenset({3, 4, 5})
C = A.symmetric_difference(B)
print(C)
```

نتیجه:

```
frozenset({1, 2, 4, 5})
```

یا:

```
C = A ^ B
```

isdisjoint()

این Method بررسی می‌کند که آیا دو مجموعه هیچ عضو مشترکی ندارند یا خیر

مثلاً:

```
A = frozenset({1, 2, 3})
B = frozenset({4, 5, 6})
print(A.isdisjoint(B))
```

خروجی:

```
True
```

چون هیچ عضو مشترکی ندارند.

اما:

```
A = frozenset({1, 2, 3})
B = frozenset({3, 4, 5})
print(A.isdisjoint(B))
```

خروجی:

```
False
```

چون 3در هر دو وجود دارد

issubset()

بررسی می‌کند که آیا یک مجموعه، زیرمجموعه‌ی مجموعه‌ی دیگر است یا خیر

```
A = frozenset({1, 2})
B = frozenset({1, 2, 3, 4})
print(A.issubset(B))
```

خروجی:

```
True
```

چون تمام اعضای Aدر Bوجود دارند می‌توانیم از =>نیز استفاده کنیم:

```
print(A <= B)
```

issuperset()

برعکس issubset()است

```
A = frozenset({1, 2, 3, 4})
B = frozenset({1, 2})
print(A.issuperset(B))
```

خروجی:

```
True
```

یعنی Aتمام اعضای Bرا در خود دارد

Proper Subset

تفاوت >با =>را نیز باید بدانیم

```
A = frozenset({1, 2})
B = frozenset({1, 2, 3})
print(A < B)
```

خروجی:

```
True
```

اما:

```
A = frozenset({1, 2})
B = frozenset({1, 2})
print(A < B)
```

خروجی:

```
False
```

چون دو مجموعه کاملاً برابر هستند

نکته بسیار مهم Frozenset :به عنوانDictionary Key

یکی از مهم‌ترین دلایل استفاده از Frozenset این است که می‌تواند به عنوان Keyدر Dictionary استفاده شود

مثلاً:

```
groups = {
   frozenset({"Python", "Django"}): "Web Development",
   frozenset({"Python", "Pandas"}): "Data Science"
}
print(groups)
```

چرا setمعمولی نمی‌تواند چنین کاری انجام دهد؟

چون Set معمولی mutableاست، اما **immutable** Frozenset است

مثلاً این کار درست نیست:

```
groups = {
   {"Python", "Django"}: "Web Development"
}
```

اما کد پایین درست است :

```
groups = {
   frozenset({"Python", "Django"}): "Web Development"}
```

مثال

فرض کنید می‌خواهیم ترکیب مهارت‌های یک گروه را به عنوان Key ذخیره کنیم:

```
skills = frozenset({"Python", "SQL", "Pandas"})
jobs = {
   skills: "Data Analyst"
}
print(jobs[skills])
```

خروجی:

Data Analyst

تمرین ۱

یک Frozenset از رنگ‌های زیر ایجاد کنید:

red

green

blue

سپس نوع آن را با type()نمایش دهید

تمرین ۲

Frozensetزیر را ایجاد کنید:

```
numbers = frozenset({10, 20, 30, 40})
```

با استفاده از forتمام اعضا را چاپ کنید

تمرین ۳

کد زیر را بررسی کنید:

```
numbers = frozenset({1, 2, 3})
numbers.add(4)
```

چرا برنامه خطا می‌دهد؟

تمرین ۴

دو Frozenset ایجاد کنید:

```
A = frozenset({1, 2, 3, 4})
B = frozenset({3, 4, 5, 6})
```

موارد زیر را پیدا کنید:

۱. Union
۲. Intersection
۳. Difference
۴. Symmetric Difference

تمرین ۵

بررسی کنید آیا مجموعه‌ی زیر زیرمجموعه‌ی مجموعه‌ی دوم است: از issubset()استفاده کنید

```
A = frozenset({1, 2})
B = frozenset({1, 2, 3, 4})
```

تمرین ۶

بررسی کنید آیا دو مجموعه هیچ عضو مشترکی ندارند:

```
A = frozenset({1, 2, 3})
B = frozenset({4, 5, 6})
```

از isdisjoint()استفاده کنید

تمرین ۷

یک Dictionary ایجاد کنید که Key آن یک Frozenset باشد:

```
key = frozenset({"Python", "Django"})
data = {
   key: "Web Developer"}
```

سپس مقدار مربوط به Key را چاپ کنید

تمرین ۸ -مقایسه Set وFrozenset

کد زیر را بررسی کنید:

```
myset = {1, 2, 3}

myfrozenset = frozenset({1, 2, 3})
```

مشخص کنید کدام یک از دستورات زیر برای هرکدام قابل اجراست:

```
myset.add(4)

myfrozenset.add(4)

myset.remove(1)

myfrozenset.remove(1)
```

Python – Set Methods

Pythonمتدهای مختلفی برای کار با setدارد. بعضی از آن‌ها Setاصلی را تغییر می‌دهند و بعضی یک Setجدید برمی‌گردانند

جدول کاملSet Methods

Method	Operator	کاربرد
add()	—	اضافه کردن یک عضو
clear()	—	حذف تمام اعضا
copy()	—	ایجاد یک کپی
discard()	—	حذف یک عضو بدون خطا در صورت نبودن
remove()	—	حذف یک عضو؛ در صورت نبودن خطا می‌دهد
pop()	—	حذف یک عضو نامشخص
union()	|	ترکیبSet ها
update()	|=	اضافه کردن اعضای Set دیگر به Set اصلی
intersection()	&	اعضای مشترک
intersection_update()	&=	نگه داشتن اعضای مشترک در Set اصلی
difference()	-	اعضای Set اول که در Set دوم نیستند
difference_update()	-=	حذف اعضای مشترک از Set اصلی
symmetric_difference()	^	اعضای غیرمشترک
symmetric_difference_update()	^=	اعمال Symmetric Difference روی Set اصلی
isdisjoint()	—	بررسی نداشتن عضو مشترک
issubset()	<= / <	بررسی زیرمجموعه بودن
issuperset()	>= / >	بررسی ابرمجموعه بودن

add()

برای اضافه کردن یک عضو به Set استفاده می‌شود

```
fruits = {"apple", "banana"}
fruits.add("orange")
print(fruits)
```

نتیجه شامل orangeخواهد بود

نکته

اگر عضو از قبل وجود داشته باشد، دوباره اضافه نمی‌شود:

```
fruits.add("apple")
```

Setهمچنان فقط یک appleخواهد داشت

update()

برای اضافه کردن چند عضو استفاده می‌شود.

```
fruits = {"apple", "banana"}
new_fruits = {"orange", "mango"}
fruits.update(new_fruits)
print(fruits)
```

حالا:

```
apple
banana
orange
mango
```

تفاوت add()و update()

fruits.add("orange") یک عضو اضافه می‌کند اما fruits.update(["orange", "mango"]) چند عضو را اضافه می‌کند

remove()

برای حذف یک عضو مشخص:

```
fruits = {"apple", "banana", "cherry"}

fruits.remove("banana")

print(fruits)
```

bananaحذف می‌شود

نکته مهم : اگر عضو وجود نداشته باشد ،fruits.remove("orange") خطا ایجاد می‌شود

discard()

برای حذف یک عضو استفاده می‌شود

```
fruits = {"apple", "banana", "cherry"}

fruits.discard("banana")

print(fruits)
```

تفاوت مهم:

fruits.discard("orange")

اگر orangeوجود نداشته باشد، خطا نمی‌دهد

مقایسه

Method	عضو وجود داشته باشد	عضو وجود نداشته باشد
remove()	حذف می‌کند	Error
discard()	حذف می‌کند	بدونError

pop()

یک عضو را از Set حذف می‌کند

```
fruits = {"apple", "banana", "cherry"}

x = fruits.pop()

print(x)

print(fruits)
```

چون Set ترتیب مشخصی ندارد، نباید انتظار داشته باشیم یک عضو خاص حذف شود.

clear()

تمام اعضای Set را حذف می‌کند:

```
fruits = {"apple", "banana", "cherry"}

fruits.clear()

print(fruits)
```

خروجی:

```
set()
```

یعنی Set خالی شده است

copy()

یک کپی از Set ایجاد می‌کند:

```
fruits = {"apple", "banana", "cherry"}

new_fruits = fruits.copy()

print(new_fruits)
```

حالا دو Set داریم:

```
fruits

new_fruits
```

و تغییر یکی، دیگری را تغییر نمی‌دهد:

```
new_fruits.add("orange")
```

```
print(fruits)
print(new_fruits)
```

union()

تمام اعضای دو Set را ترکیب می‌کند:

```
A = {1, 2, 3}
B = {3, 4, 5}
C = A.union(B)
print(C)
```

نتیجه:

```
{1, 2, 3, 4, 5}
```

یا:

```
C = A | B
```

intersection()

فقط اعضای مشترک را برمی‌گرداند:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
C = A.intersection(B)
print(C)
```

نتیجه:

```
{3, 4}
```

یا:

```
C = A & B
```

intersection_update()

مشابه intersection()است، اما Setاصلی را تغییر می‌دهد

```
A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

A.intersection_update(B)

print(A)
```

نتیجه:

```
{3, 4}
```

تفاوت

```
C = A.intersection(B)
```

یک Set جدید می‌سازد

اما A.intersection_update(B) خود Aرا تغییر می‌دهد

difference()

اعضایی را برمی‌گرداند که در Set اول هستند ولی در Set دوم نیستند

```
A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

C = A.difference(B)

print(C)
```

نتیجه:

```
{1, 2}
```

یا:

```
C = A – B
```

difference_update()

Setاصلی را تغییر می‌دهد:

```
A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

A.difference_update(B)

print(A)
```

نتیجه:

```
{1, 2}
```

symmetric_difference()

اعضای غیرمشترک دو Set را برمی‌گرداند

```
A = {1, 2, 3}

B = {3, 4, 5}

C = A.symmetric_difference(B)

print(C)
```

نتیجه:

```
{1, 2, 4, 5}
```

3حذف شده چون در هر دو Set وجود دارد

یا C = A ^ B

symmetric_difference_update()

مانند قبلی است، ولی Aرا تغییر می‌دهد:

```
A = {1, 2, 3}

B = {3, 4, 5}

A.symmetric_difference_update(B)
```

```
print(A)
```

نتیجه:

```
{1, 2, 4, 5}
```

isdisjoint()

بررسی می‌کند آیا دو Set هیچ عضو مشترکی ندارند

```
A = {1, 2, 3}
B = {4, 5, 6}
print(A.isdisjoint(B))
```

نتیجه:

```
True
```

اما:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A.isdisjoint(B))
```

نتیجه:

```
False
```

چون 3مشترک است

issubset()

بررسی می‌کند آیا تمام اعضای یک Set در Set دیگر وجود دارند یا خیر

```
A = {1, 2}
B = {1, 2, 3, 4}
print(A.issubset(B))
```

نتیجه:

```
True
```

می‌توانیم از =>نیز استفاده کنیم:

```
print(A <= B)
```

issuperset()

برعکس issubset()است

```
A = {1, 2, 3, 4}
B = {1, 2}
print(A.issuperset(B))
```

نتیجه:

```
True
```

یا:

```
print(A >= B)
```

>و <چه تفاوتی دارند؟

این قسمت برای دانش‌آموزان کمی مهم‌تر است

```
A = {1, 2}
B = {1, 2, 3}
print(A < B)
```

نتیجه:

```
True
```

چون Aیک Proper Subsetاز Bاست

اما print(A <= B) نیز Trueاست

تفاوت زمانی مشخص می‌شود که دو Set برابر باشند:

```
A = {1, 2}
B = {1, 2}
print(A <= B)  # True
print(A < B)   # False
```

دسته‌بندی Methods

اضافه کردن

add()

update()

حذف کردن

remove()

discard()

pop()

clear()

کپی

copy()

ترکیب و مقایسه

union()

intersection()

difference()

symmetric_difference()

نسخه‌های Update

intersection_update()

difference_update()

symmetric_difference_update()

بررسی رابطه

isdisjoint()

issubset()

issuperset()

تفاوت بسیار مهم Return :یا تغییر Set اصلی؟

این متدها معمولاً Set جدید برمی‌گردانند:

union()

intersection()

difference()

symmetric_difference()

مثلاً:

A = {1, 2, 3}

B = {3, 4, 5}

C = A.union(B)

Aهمچنان بدون تغییر است

اما این متدها Set اصلی را تغییر می‌دهند:

update()

intersection_update()

difference_update()

symmetric_difference_update()

مثلاً:

A = {1, 2, 3}

B = {3, 4, 5}

A.update(B)

حالا خود Aتغییر کرده است

Method	Setاصلی را تغییر می‌دهد؟	نتیجه
add()	✓	None
update()	✓	None
remove()	✓	None
discard()	✓	None
pop()	✓	عضو حذف‌شده
clear()	✓	None
copy()	✗	Setجدید
union()	✗	Setجدید
intersection()	✗	Setجدید
difference()	✗	Setجدید
symmetric_difference()	✗	Setجدید
intersection_update()	✓	None
difference_update()	✓	None
symmetric_difference_update()	✓	None
isdisjoint()	✗	True/False
issubset()	✗	True/False
issuperset()	✗	True/False

تمرین ۱

fruits = {"apple", "banana", "cherry"}

با استفاده از ()add، orangeرا اضافه کنید

تمرین ۲

با استفاده از ()remove، bananaرا حذف کنید

تمرین ۳

بررسی کنید تفاوت ()removeو ()discardچیست

این دو را امتحان کنید:

fruits.remove("orange")

و:

fruits.discard("orange")

چه اتفاقی می‌افتد؟

تمرین ۴

دو Set زیر را دارید:

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

نتیجه‌ی موارد زیر را پیدا کنید:

A | B

A & B

A - B

A ^ B

تمرین ۵

برنامه‌ای بنویسید که:

python = {"Ali", "Sara", "Reza"}

java = {"Ali", "Reza", "Hassan"}

مشخص کند چه دانش‌آموزانی در هر دو کلاس هستند

تمرین ۶

بررسی کنید آیا:

A = {1, 2}

B = {1, 2, 3, 4}

Aزیرمجموعه‌ی Bاست یا خیر

تمرین ۷

دو Set زیر را در نظر بگیرید:

online_students = {"Ali", "Sara", "Reza", "Maryam"}

```
python_students = {"Ali", "Reza", "Hassan"}
```

برنامه‌ای بنویسید که:

۱. دانشجویان مشترک را پیدا کند

۲. دانشجویانی که فقط در Python هستند را پیدا کند

۳. تمام دانشجویان را پیدا کند

۴. بررسی کند آیا دو کلاس هیچ دانشجوی مشترکی ندارند یا خیر

تمرین چالشی

برنامه‌ای برای مدیریت محصولات فروشگاه بنویسید:

```
shop = {"laptop", "mouse", "keyboard"}
```

برنامه باید:

۱. یک محصول جدید اضافه کند

۲. یک محصول را حذف کند

۳. محصولات موجود را نمایش دهد

۴. بررسی کند آیا laptopموجود است

۵. یک Set دیگر از محصولات فروشگاه دوم دریافت کند

۶. محصولات مشترک دو فروشگاه را نمایش دهد

۷. تمام محصولات دو فروشگاه را نمایش دهد